

Somatic variant calling & Annotation

Massimo Maiolo

Lesson plan

17.09.2026	Panos - Sofia	Introduction/Logistics + Introduction to cancer biology
24.09.2026	Philip	Introduction to software for mutation calling, gene signatures
01.10.2026	Sofia	Cancer Biology
08.10.2026	Max	Somatic variant calling
15.10.2026	Max	Somatic variant calling + variant annotation
22.10.2026	Philip	Variant prioritization - Driver mutations
29.10.2026	Philip	Mutational Signatures
05.11.2026	Andrew	Mid-term exam
12.11.2026	Panos - Andrew	Introduction to software for structural analysis
19.11.2026	Panos - Andrew	CNV calling
26.11.2026	Panos - Andrew	Structural variants
03.12.2026	Panos - Andrew	Tumor clonality
10.12.2026	Panos - Andrew	Data interpretation + summary
17.12.2026	All	Summary - Q&As
13.01.2027	Panos - Andrew	Final Exam

Github Cancer Genomics Course



<https://github.com/supermax1234/CancerGenomicsCourse>

Lesson 1: Theory

All theoretical content is available in the following PDF:

- [var_calling.pdf](#)

Lesson 2: Exercises

A step-by-step guide for completing the practical exercises is available here:

 <https://supermax1234.github.io/CancerGenomicsCourse/>

IGV Tutorial Files

The following files required for the IGV tutorial can be found in the `Exercises/IGV` folder:

```
Exercises/  
├── IGV/  
│   ├── HCC1143.normal.21.19M-20M.bam  
│   └── HCC1143.normal.21.19M-20M.bam.bai
```



All necessary files for the **Variant Calling and Filtering** exercises can be downloaded from this Google Drive link:

 [Download all files](#)

BIOINFORMATICS EXERCISES

SELECT A MODULE TO BEGIN THE PRESENTATION

1. **Variant Calling:** Techniques for identifying genetic changes (SNPs, Indels).
2. **Mutational Signatures:** Deconvolution of mutational processes in cancer genomes.

TUTORIAL OUTLINE

1. **Setup**
2. **IGV Tutorial:** A Guide to Visualizing NGS Data
3. **Data Quality, Alignment & Preprocessing:**
 - Initial QC (`FastQC`, `fastp`, `MultiQC`)
 - Read Alignment (`bwa-mem2`, `SAM` to `BAM`)
 - BAM Processing (`MarkDuplicates`, `BAM` metrics)
 - Quality Calibration (`BaseRecalibrator`, `ApplyBQSR`)
4. **Variant Calling:** The Art of Finding Genetic Changes
5. **Variant Annotation and Filtering:** Turning Variants into Insights

Course Assignment: Group Organization

Teamwork & Structure

- Organize yourselves into **groups of 3 students** (10 groups total)
- Do not concentrate all the IT skills in the same group
- The group spokesperson should come and provide me with the members' names

1	Role / Status	Last Name	First Name	Login	E-Mail
2	Member	Ambrogetti	Isaac	i.ambrogetti	isaac.ambrogetti@students.unibe.ch
3	Member	Amos	Mario	m.amos1	mario.amos@students.unibe.ch
4	Member	Bharatia	Rohit Mohan	r.bharatia	rohit.bharatia@students.unibe.ch
5	Member	Ercan	Gülce Berfin	g.ercan	guelce.ercan@students.unibe.ch
6	Member	Jakobsson	Emma Alexandra	e.jakobsson	emma.jakobsson@students.unibe.ch
7	Member	Kaufmann	Dominik Julien	d.kaufmann20	dominik.kaufmann3@students.unibe.ch
8	Member	Keane	Stephen Vincent	s.keane	stephen.keane@students.unibe.ch
9	Member	Kim	Jeehee	j.kim13	jeehee.kim@students.unibe.ch
0	Member	Kiran	Varshaa	v.kiran	varshaa.kiran@students.unibe.ch
1	Member	Krasniqi	Edrinë	e.krasniqi6	edrine.krasniqi@students.unibe.ch
2	Member	Kummer	Mario Lucio	m.kummer5	mario.kummer@students.unibe.ch
3	Member	Kunjal	Pranamyia	p.kunjal	pranamyia.kunjal@students.unibe.ch
4	Member	Li	Jiajing	j.li10	jiajing.li@students.unibe.ch
5	Member	Lin	Sheryl	s.lin2	sheryl.lin@students.unibe.ch

Course Assignment: Group Organization

Teamwork & Structure

- **Work Together:**
 - Team members share the exact **same grade** for this assignment, so **collaborate** closely
 - review each other's scripts, and
 - run through the pipeline as a team

ASSESSING RAW FASTQ QUALITY

FastQC

`FastQC` provides a comprehensive quality overview of sequencing data. It serves to detect potential artifacts, low-quality bases, or adapter contamination early in the pipeline, and is an excellent tool for an initial inspection before proceeding to alignment.

Exercise:

Your task is to write two SLURM submission scripts to run `FastQC` on the raw FASTQ files for both the normal and tumor samples. You will need to generate four reports: two for the normal sample (`R1`, `R2`) and two for the tumor sample (`R1`, `R2`).

- **Input:** *normal_R*.fastq.gz* & *tumor_R*.fastq.gz* (under `/data/courses/cancergenomics/VAR_CALLING/fastq`)
- **Output:** HTML reports and zip archives for each file

NEED A HINT? (BASH TEMPLATE)

```
1 #!/bin/bash
2
3 R1=normal_R1.fastq.gz
4 R2=normal_R2.fastq.gz
5 singularity exec \
6   --bind /storage \
7   depot.galaxyproject.org-singularity-fastqc-0.11.9--0.img \
8   fastqc -t 4 $R1 $R2
```

CLUSTER ENVIRONMENT & SLURM GUIDELINES

Cluster Environment & Job Submission

- **No Running on the Master Node:** Never run heavy compute jobs, alignments, or variant calling directly on the login/master node. Always submit your jobs to the cluster queue using `SLURM` (`sbatch`).
- **Standard SLURM Headers:** Use the correct template depending on the session date:
Session 1 (Oct 8, 2026):

```
1 #!/bin/bash
2 #SBATCH --job-name=class_assignment
3 #SBATCH --partition=pibu_el8
4 #SBATCH --nodes=1
5 #SBATCH --ntasks-per-node=4
6 #SBATCH --mem=32G
7 #SBATCH --time=02:00:00
8 #SBATCH --account=class-407009-ws-2026
9 #SBATCH --reservation=class-407009-2026-10-08
```



Course Assignment: Submission Guidelines

Submission Details

- **Format:** Submit (email: massimo.maiolo@unibe.ch) a **single compressed .zip** (or *tar.gz*) folder containing:
 - all your developed **SLURM** scripts, following the sequential numbering convention (e.g., *01_*, *02_*, etc.) covered in class
 - A **README** text file containing the full names and surnames of the 3 group members. Any notes, comments, or explanations in the README if a specific step failed, threw warnings, or required alternative handling
- **Deadline:** submit your group's work by **October 25** (23:59)

Important Reminder: SLURM Script Requirements

SLURM scripts vs. Local bash

- Every script you submit must be a fully functional, runnable **SLURM batch script**, not a local bash script
- Each script must start with the required *#SBATCH* header (partition, memory, time, job name, and reservation) as shown in the course template
- **Never** run heavy compute tasks directly on the master/login node; always submit your scripts via `sbatch`

Slurm Cheatsheet

<https://www.carc.usc.edu/user-guides/hpc-systems/using-our-hpc-systems/slurm-cheatsheet>

Job management

squeue - View information about jobs in scheduling queue ([docs](#))

Option	Description
-A, -account=<account_list>	Filter by accounts (comma-separated list)
-o, -format=<options>	Output format to display
-j, -jobs=<job_id_list>	Filter by job IDs (comma-separated list)
-l, -long	Show more available information
-me	Filter by your own jobs
-n, -name=<job_name_list>	Filter by job names (comma-separated list)
-p, -partition=<partition_list>	Filter by partitions (comma-separated list)
-P, -priority	Sort jobs by priority
-start	Show the expected start time and resources to be allocated for pending jobs
-t, -states=<state_list>	Filter by states (comma-separated list)
-u, -user=<user_list>	Filter by users (comma-separated list)

HPC Environment Modules – Essential Commands

Before running tools or writing your scripts, you need to manage your software environment using the **module** system.

Here are the 4 most important commands:

- **1. Search for available software**
 - `module avail [tool_name]`: Lists all available modules or filters by a specific tool name
- **2. Find tools across all versions/categories:**
 - `module spider <tool_name>`: Extremely useful if you are unsure of the exact module name or version
- **3. Load a module into your session:**
 - `module load <module_name>`: Activates the tool so it is available in your PATH.
- **4. Check currently loaded modules:**
 - `module list`: Verifies what software is currently active in your environment

Missing Modules & Container Fallback

What to do if a module is missing or fails?

If a required software module is not available or broken on the cluster, you can download a pre-compiled container image directly from **Docker Hub**, **Quay.io**, or **Bioconda**

Downloading an Image via Singularity/Apptainer

Pull a container image from Bioconda / Quay registry

```
singularity pull docker://quay.io/biocontainers/<tool_name>:<version>
```

Or from standard Docker Hub

```
singularity pull docker://<repository>/<image>:<tag>
```

Once downloaded, you can run it inside your SLURM scripts using:

```
singularity exec --bind /storage image_file.sif <command>
```



Happy coding!